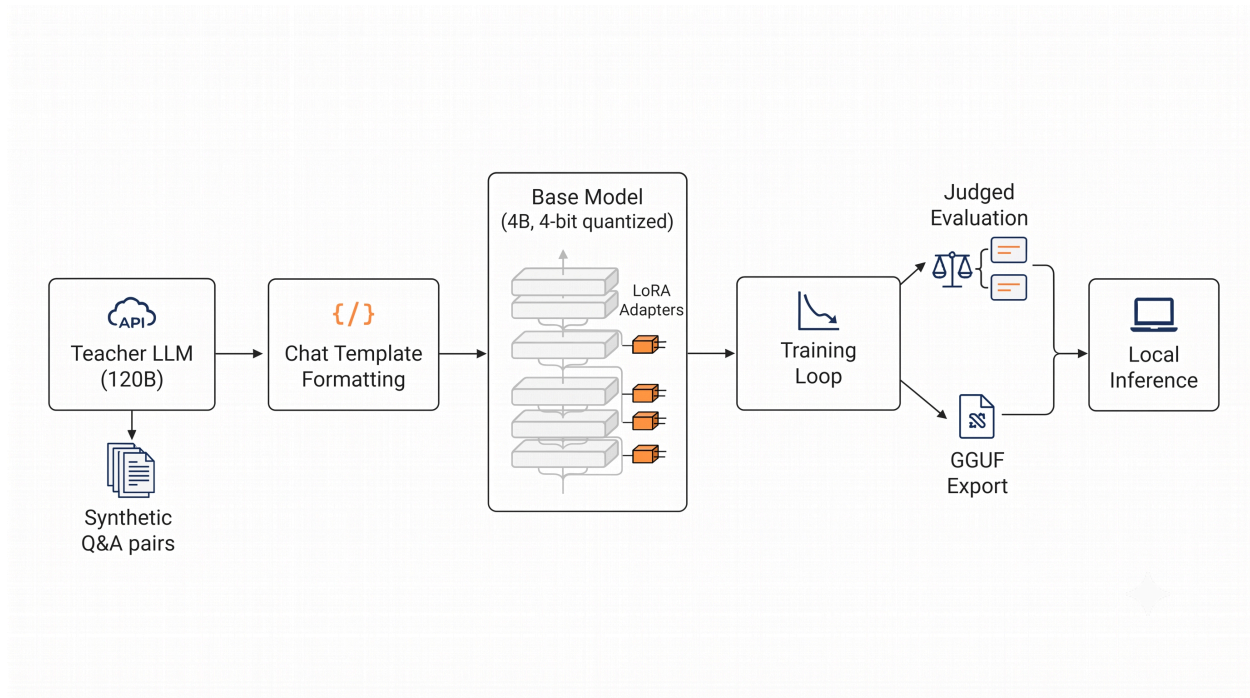


# Fine-Tune Your Own LLM for Free on a Kaggle GPU - in 30 Minutes

Created by <https://www.instagram.com/datasciencebrain/>



Verified 2026-07-08 - unsloth==2026.7.1, transformers==4.56.2, trl==0.22.2, torch (cu128), groq (openai/gpt-oss-120b), base model Qwen3-4B-Instruct-2507, Kaggle GPU T4 ×2, Python 3.11 (Kaggle default).

## Table of Contents

1. What You're Building
2. The Core Idea, in Plain English
3. Why These Two Models
4. Architecture
5. Kaggle Setup
6. Cell 1 - Install the Stack
7. Cell 2 - Generate the Dataset with Groq
8. Cell 3 - Load Qwen3-4B in 4-bit and Attach LoRA
9. Cell 4 - Format the Data with the Chat Template
10. Cell 5 - Baseline: How the Base Model Talks

11. Cell 6 - Train
  12. Cell 7 - Talk to Your Fine-Tuned Model
  13. Cell 8 - Prove It Beats the Base Model
  14. Cell 9 - Save the Adapters and Export GGUF
  15. Run It Locally with Ollama
  16. Common Errors & Fixes
  17. Knowledge Check
  18. What You Should Remember
  19. Key Terms Learned
  20. What to Build Next
- 

## 1. What You're Building

You're fine-tuning a real 4-billion-parameter LLM - **Qwen3-4B-Instruct** - into **DesiTutor**: an AI-engineering tutor that answers every question in natural Hinglish, always drops one desi analogy (cricket, chai, local trains), and always ends with a one-line English summary. No system prompt needed. The persona lives **inside the weights**.

**The full loop, all free.** A large model on Groq's free tier ( `openai/gpt-oss-120b` , 120B parameters) writes your training dataset. A free Kaggle T4 GPU trains the small model on it. This is called **distillation** - a big teacher model generating data to teach a small student model - and it's how most production fine-tunes are actually built.

**The user experience.** After training, you ask the model "What is overfitting?" and instead of a textbook English paragraph, you get:

```
Dekho, overfitting matlab model ne training data ko ratta maar liya hai -  
samajh kuch nahi aaya. Jaise woh topper jo guess paper ke saare answers  
yaad kar leta hai, par exam mein question ghuma diya toh blank ho jaata hai...  
Bottom line: An overfit model memorizes training data instead of learning patterns.
```

The base model cannot do this consistently, no matter how you prompt it across a long conversation. Your fine-tuned one does it every single time - that's the before/after you'll measure with a judged eval at the end.

**Why this skill pays.** Fine-tuning (PEFT/LoRA) is the highest-compensated specialization in applied AI right now, because it's the skill that turns a generic model into a company's *own* model - their tone, their format, their domain. Everyone can call an API. Far fewer people can produce a model checkpoint that beats the base model on a custom eval. After this guide, you'll have done exactly that, end to end, for ₹0.

---

## 2. The Core Idea, in Plain English

### Fine-tuning vs Prompting vs RAG

#### Plain English

Prompting *tells* the model how to behave, every single time. RAG *hands* the model facts to read, every single time. Fine-tuning *changes the model itself*, once.

## Analogy

Prompting is giving an actor stage directions before every scene. RAG is handing them a script to read from. Fine-tuning is sending them to acting school - after that, the behaviour is just who they are.

## Technical Explanation

Fine-tuning continues training a pretrained model on your own examples, updating weights so the target behaviour becomes the model's default. Use it for **style, format, tone, and domain-specific behaviour** - things that must hold on every response. Use RAG when the problem is missing *knowledge* (facts change; retraining for facts is wasteful). The two are complementary, not competitors.

## Project Usage

Here you fine-tune for persona and format: Hinglish + analogy + "Bottom line:" ending. That's a behaviour, not a fact - the perfect fine-tuning target.

## LoRA

### Plain English

Instead of retraining all 4 billion weights, LoRA freezes the model and trains small add-on matrices - like clipping a lens onto a camera instead of rebuilding the camera.

### Analogy

Editing a 1,000-page book by attaching sticky notes on the pages that matter, instead of reprinting the whole book. The book (base model) stays untouched; the sticky notes (adapters) carry your changes and fit in your pocket.

### Technical Explanation

**LoRA (Low-Rank Adaptation)** freezes the base weights  $W$  and learns an update  $\Delta W = B \cdot A$ , where  $A$  and  $B$  are two thin matrices of rank  $r$  (we use  $r=32$ ). Only  $A$  and  $B$  train. For Qwen3-4B that's roughly 66M trainable parameters out of 4B - about 1.6%, and the training signal that actually matters flows through far less. The output is a tiny adapter file (~200 MB) instead of a full 8 GB model copy.

Full fine-tuning of even a 7B model needs 80GB-class GPUs and serious money. LoRA on a free 16 GB T4 gets you 90%+ of the quality for style/format tasks.

## QLoRA

### Plain English

QLoRA is LoRA with one extra trick: the frozen base model is compressed to 4-bit numbers so it fits in a small GPU.

## Technical Explanation

The frozen weights are stored in 4-bit (via `bitsandbytes`), cutting base-model memory ~4x, while the LoRA adapters still train in 16-bit precision. Quality loss from quantizing the *frozen* part is minimal because the *trainable* part stays full precision. This is why a 4B model + optimizer + activations fits inside a T4's 16 GB.

## Project Usage

`load_in_4bit = True` in Cell 3 is the entire QLoRA switch. Unsloth handles the rest and also makes training ~2x faster than vanilla HuggingFace.

## 3. Why These Two Models

**Base model:** `unsloth/Qwen3-4B-Instruct-2507`

Three candidates fit a T4: Llama-3.2-3B, Gemma-3-4B, and Qwen3-4B. Qwen3-4B-Instruct-2507 wins on every axis that matters for this project:

| Criterion                       | Qwen3-4B-Instruct-2507                                | Llama-3.2-3B                | Gemma-3-4B               |
|---------------------------------|-------------------------------------------------------|-----------------------------|--------------------------|
| Gated on HuggingFace?           | <b>No - zero friction</b>                             | Yes (license approval wait) | Yes (license acceptance) |
| Fine-tuned quality in class     | <b>Best</b> - tops small-model fine-tuning benchmarks | Good                        | Good                     |
| Hindi/Hinglish ability          | <b>Strong</b> (multilingual-first training)           | Weak-moderate               | Strong                   |
| Thinking-token complications    | <b>None</b> (Instruct variant is non-thinking)        | None                        | None                     |
| Official tested Kaggle notebook | <b>Yes</b> (Unsloth)                                  | Yes                         | Yes                      |

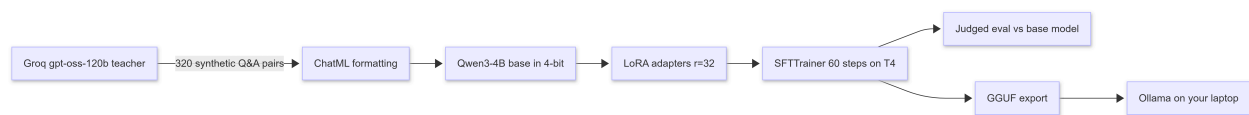
The gating point matters more than it sounds: with Llama or Gemma, every reader must create a HuggingFace account, request access, wait for approval, and wire up a token before step one. With Qwen, the model just downloads. The `-2507` suffix is the July-2025 refresh - noticeably stronger than original Qwen3-4B - and the **Instruct** (non-thinking) variant means no `<think>` blocks polluting your training data.

**Data-generation model:** `openai/gpt-oss-120b` on Groq

The teacher model needs to be much bigger and smarter than the student, free, and fast. `openai/gpt-oss-120b` on Groq is all three: 120B parameters, free tier with no credit card, and Groq's speed means 320 training examples in ~2 minutes.

One warning if you've followed earlier guides in this series: **do not use** `llama-3.3-70b-versatile` **here**. Groq is shutting it down on August 16, 2026, and `openai/gpt-oss-120b` is Groq's official recommended replacement. Any notebook built on the Llama model stops working next month.

## 4. Architecture



The flow in one breath: the big Groq model writes 320 DesiTutor-style Q&A pairs → each pair is formatted into Qwen's chat template → the 4B base model loads in 4-bit → LoRA adapters attach on top → the trainer runs 60 steps (~12 minutes on a T4) → a judged eval proves the tuned model beats the base → the result exports as a GGUF file you can run on your own laptop with Ollama.

Time budget: install ~4 min, data generation ~3 min, model load ~2 min, training ~12 min, eval ~4 min, saving ~2 min. Total ≈ 27 minutes of runtime. The optional GGUF export adds ~10 more.

## 5. Kaggle Setup

Everything runs in one Kaggle notebook. Four things to set up before writing any code.

- 1. Account + phone verification.** Sign up at [kaggle.com](https://kaggle.com). Then go to Settings → Phone verification and verify your number - Kaggle requires this to unlock GPUs and internet access in notebooks. Takes two minutes.
- 2. Create the notebook and pick the GPU.** Click **Create** → **New Notebook**. In the right-hand panel: **Settings** → **Accelerator** → **GPU T4 ×2**. You get 30 GPU-hours per week (resets Saturday midnight UTC); this project uses well under one hour. Unsloth trains on a single GPU, so T4 ×2 costs the same quota as one T4.
- 3. Turn internet on.** Same right-hand panel: **Settings** → **Internet** → **On**. Without this, the notebook can't download the model or call Groq, and you'll see connection errors that look like library bugs.
- 4. Add your Groq key as a secret.** Get a free key at [console.groq.com/keys](https://console.groq.com/keys) (no credit card). In the notebook: **Add-ons** → **Secrets** → **Add secret**, name it exactly `GROQ_API_KEY`, paste the key, and tick the checkbox to attach it to this notebook. Never paste keys into notebook cells - Kaggle notebooks are public by default when shared.

## 6. Cell 1 - Install the Stack

Kaggle's base image ships its own PyTorch, and the single most common way fine-tuning notebooks die is letting `pip` silently upgrade or downgrade torch, which breaks `xformers` (compiled against one exact torch version). This install sequence is Unsloth's officially tested Kaggle recipe: it installs a matched torch + xformers set from the PyTorch index first, then Unsloth, then pins `transformers` and `trl` to the versions Unsloth is validated against. Run it exactly as written, as the very first cell, and don't add or upgrade anything after it.

```
%%capture
import os

!pip install pip3-autoremove
!pip install torch torchvision torchaudio xformers --index-url https://download.pytorch.org/whl/cu128
!pip install unsloth
!pip install --no-deps --upgrade "torchao>=0.16.0"
!pip install transformers==4.56.2
```

```
!pip install --no-deps trl==0.22.2
!pip install groq
```

This takes 3–5 minutes. `%%capture` hides the wall of pip output; if a later import fails, re-run this cell **without** the `%%capture` line to see what actually went wrong.

The last line matters too: Kaggle's base image does **not** include the `groq` SDK, so without it Cell 2 dies on `ModuleNotFoundError: No module named 'groq'`. It's a pure-API client (no torch dependency), so it's safe to install after the pinned stack.

The `trl` pin matters more than it looks: TRL's API changed hard across versions (`tokenizer=` was removed, `max_seq_length` was renamed), and most tutorial code on the internet targets a dead signature. Pinning `0.22.2` means the code in this guide matches the library you're running.

## 7. Cell 2 - Generate the Dataset with Groq

Fine-tuning quality is 80% dataset quality, so this is the most important cell in the notebook. The strategy: give the teacher model a precise persona spec - the three rules DesiTutor must follow - and ask for question-answer pairs across 16 different AI-engineering topics, 20 per topic. Topic spread prevents the classic failure where the model learns the style but only for questions about, say, neural networks.

First, the client and the persona spec. `AsyncGroq` is the native async client, and Jupyter supports `await` directly in cells:

```
import asyncio, json, random
from groq import AsyncGroq
from kaggle_secrets import UserSecretsClient

client = AsyncGroq(api_key=UserSecretsClient().get_secret("GROQ_API_KEY"))

TOPICS = [
    "Python basics and data structures", "pandas and data cleaning",
    "machine learning fundamentals", "overfitting and regularization",
    "neural networks and backpropagation", "transformers and attention",
    "LLMs and how they are trained", "prompt engineering",
    "RAG and vector databases", "fine-tuning and LoRA",
    "AI agents and tool calling", "model evaluation and metrics",
    "APIs and deployment basics", "SQL and databases",
    "statistics for data science", "career advice for AI engineers",
]

SYSTEM_PROMPT = """You generate training data for 'DesiTutor', an AI tutor for Indian developers.
Each example is a question a beginner Indian developer would ask, plus DesiTutor's answer.
Every answer MUST follow these 3 rules:
1. Written in natural Hinglish (Roman script, Hindi-English code-mixed, like Indian devs actually talk).
2. Contains exactly one relatable desi analogy (chai, cricket, local trains, tiff
```

```

in, shaadi, auto-rickshaw, etc).
3. Ends with a one-line English summary starting with 'Bottom line:'.
Keep answers 60-120 words. Vary question phrasing and difficulty.
Return JSON: {"examples": [{"user": "...", "assistant": "..."}, ...]}"""

```

Now the generator. One request per topic, JSON mode on, with a retry loop - and the concurrency number is not arbitrary. Groq's free tier allows **8,000 tokens per minute** on this model, and one 20-pair request costs ~3,300 tokens, so two concurrent requests (~6,700) is the most that fits under the cap. The retry loop is still there because the TPM window slides and an occasional `429` (or a transient `json_validate_failed` from JSON mode) mid-run shouldn't kill the dataset:

```

async def generate_batch(topic: str, n: int = 20, retries: int = 3) -> list[dict]:
    """Ask the teacher model for n Q&A pairs about one topic. Backs off on rate limits."""
    for attempt in range(retries):
        try:
            resp = await client.chat.completions.create(
                model="openai/gpt-oss-120b",
                messages=[{"role": "system", "content": SYSTEM_PROMPT},
                          {"role": "user", "content": f"Generate {n} question-answer pairs about: {topic}"}],
                response_format={"type": "json_object"},
                temperature=0.9,
            )
            return json.loads(resp.choices[0].message.content)["examples"]
        except Exception as e:
            print(f"  retry {attempt + 1} for '{topic}': {e}")
            await asyncio.sleep(20 * (attempt + 1))
    return []

async def build_dataset() -> list[dict]:
    """Generate all topics, 2 at a time - free tier allows 8,000 tokens/min, one call uses ~3,300."""
    pairs = []
    for i in range(0, len(TOPICS), 2):
        results = await asyncio.gather(*[generate_batch(t) for t in TOPICS[i:i + 2]])
        for batch in results:
            pairs.extend(batch)
        print(f"{len(pairs)} examples generated")
    return pairs

pairs = await build_dataset()
random.shuffle(pairs)

```

Finally, split off 20 pairs the model will **never see during training** - that's the held-out set the eval in Cell 8 uses - and save everything to disk so a session restart doesn't cost you the Groq calls:

```
train_pairs, heldout_pairs = pairs[:-20], pairs[-20:]

with open("/kaggle/working/desitutor_data.json", "w") as f:
    json.dump({"train": train_pairs, "heldout": heldout_pairs}, f, ensure_ascii=False, indent=2)

print(f"Train: {len(train_pairs)}, Held-out: {len(heldout_pairs)}")
print(json.dumps(train_pairs[0], ensure_ascii=False, indent=2))
```

**What to verify:** you should end with roughly 280–320 pairs total (the model doesn't always return exactly 20 per call), minus 20 held-out. A few `retry ...` lines in the output are normal - that's the rate limiter doing its job, not a failure. The whole cell takes 3–5 minutes. Read the printed sample - the answer must be Hinglish, contain an analogy, and end with "Bottom line:". If the samples look weak, your model will be weak; fix the prompt before training, not after.

## 8. Cell 3 - Load Qwen3-4B in 4-bit and Attach LoRA

Two steps happen here. First, `FastLanguageModel.from_pretrained` downloads the base model and loads it with 4-bit quantization - the QLoRA half. Second, `get_peft_model` freezes everything and injects trainable LoRA matrices into the seven weight matrices that matter in a transformer block: the four attention projections (`q,k,v,o`) and the three MLP projections (`gate,up,down`).

One rule that trips people up: `from unsloth import ...` **must be the first ML import in the notebook** - Unsloth patches transformers at import time, so importing transformers first disables the speedups.

```
from unsloth import FastLanguageModel

model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Qwen3-4B-Instruct-2507",
    max_seq_length = 2048,
    load_in_4bit = True,
    load_in_8bit = False,
    full_finetuning = False,
)

model = FastLanguageModel.get_peft_model(
    model,
    r = 32,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                     "gate_proj", "up_proj", "down_proj"],
    lora_alpha = 32,
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = "unsloth",
```

```

    random_state = 3407,
    use_rslora = False,
    loftq_config = None,
)

```

The knobs, briefly: `r = 32` is the adapter rank - higher learns more but risks overfitting a small dataset; 16-32 is the sweet spot for style transfer. `lora_alpha = 32` scales the adapter's influence (rule of thumb: alpha = r). `use_gradient_checkpointing = "unsloth"` trades a little compute for a lot of VRAM, which is what lets 2048-token sequences train on a T4.

**What to verify:** the download is ~4 GB and takes a couple of minutes. Unsloth prints a trainable-parameters line - expect roughly 66M trainable out of ~4B total (~1.6%). That single printed line is the whole LoRA pitch: you are training under 2% of the model.

## 9. Cell 4 - Format the Data with the Chat Template

An instruct model doesn't see raw text - it sees a conversation serialized in its **chat template**. Qwen uses ChatML: each turn wrapped in `<|im_start|>role ... <|im_end|>` markers. Training data must be serialized with the *exact same* template the model saw during its original training, or you're teaching it a foreign language. `apply_chat_template` does this correctly so you never hand-build those strings.

Note what's *not* here: no system prompt. DesiTutor's persona comes purely from the assistant answers - after training, the behaviour appears with zero prompting.

```

from unsloth.chat_templates import get_chat_template
from datasets import Dataset

tokenizer = get_chat_template(tokenizer, chat_template = "qwen3-instruct")

def to_conversation(pair: dict) -> dict:
    """Wrap one Q&A pair in the role/content structure chat templates expect."""
    return {"conversations": [
        {"role": "user", "content": pair["user"]},
        {"role": "assistant", "content": pair["assistant"]},
    ]}

def formatting_prompts_func(examples: dict) -> dict:
    """Serialize each conversation into one ChatML training string."""
    texts = [tokenizer.apply_chat_template(c, tokenize = False, add_generation_prompt = False)
              for c in examples["conversations"]]
    return {"text": texts}

dataset = Dataset.from_list([to_conversation(p) for p in train_pairs])
dataset = dataset.map(formatting_prompts_func, batched = True)
print(dataset[0]["text"])

```

**What to verify:** the printed sample must show `<|im_start|>user`, the question, `<|im_start|>assistant`, and the Hinglish answer, ending with `<|im_end|>`. If you see raw JSON or missing markers, the template didn't apply.

## 10. Cell 5 — Baseline: How the Base Model Talks

Before training, capture what the untrained model says to the held-out questions. This is your "before" photo — Cell 8 compares it against the "after". Running this now (with adapters already attached) is still a true baseline: freshly initialized LoRA adapters are mathematically a no-op — the `B` matrix starts at zero, so  $\Delta W = B \cdot A = 0$  and outputs are identical to the raw base model.

The sampling settings (`temperature=0.7, top_p=0.8, top_k=20`) are Qwen's official recommendation for this model — keep them for every generation in this notebook.

```
from transformers import TextStreamer

def ask(question: str, max_new_tokens: int = 200) -> str:
    """Generate one answer from the model in its current state."""
    messages = [{"role": "user", "content": question}]
    text = tokenizer.apply_chat_template(messages, tokenize = False, add_generation_prompt = True)
    inputs = tokenizer(text, return_tensors = "pt").to("cuda")
    out = model.generate(**inputs, max_new_tokens = max_new_tokens,
                        temperature = 0.7, top_p = 0.8, top_k = 20)
    return tokenizer.decode(out[0][inputs["input_ids"].shape[1]:], skip_special_tokens = True)

heldout_questions = [p["user"] for p in heldout_pairs]
base_answers = [ask(q) for q in heldout_questions]

print("Q:", heldout_questions[0])
print("BASE MODEL:", base_answers[0][:400])
```

**What to verify:** the base model answers in polished, formal English. No Hinglish, no chai analogies, no "Bottom line:". Generating 20 answers takes 2–3 minutes. Keep this cell's output visible — the contrast after training is the money shot.

## 11. Cell 6 — Train

`SFTTrainer` (Supervised Fine-Tuning Trainer) from TRL runs the actual training loop, and Unsloth accelerates it under the hood. One extra trick here does outsized work: `train_on_responses_only` masks the loss on user turns, so the model is only graded on the *assistant's* words. Without it, the model wastes capacity learning to predict your questions.

```
from trl import SFTTrainer, SFTConfig
from unsloth.chat_templates import train_on_responses_only

trainer = SFTTrainer(
```

```

model = model,
tokenizer = tokenizer,
train_dataset = dataset,
eval_dataset = None,
args = SFTConfig(
    dataset_text_field = "text",
    per_device_train_batch_size = 2,
    gradient_accumulation_steps = 4,
    warmup_steps = 5,
    max_steps = 60,
    learning_rate = 2e-4,
    logging_steps = 1,
    optim = "adamw_8bit",
    weight_decay = 0.001,
    lr_scheduler_type = "linear",
    seed = 3407,
    report_to = "none",
),
)

trainer = train_on_responses_only(
    trainer,
    instruction_part = "<|im_start|>user\n",
    response_part = "<|im_start|>assistant\n",
)

stats = trainer.train()

```

The math behind the numbers: batch 2 × gradient accumulation 4 = effective batch of 8. 60 steps × 8 = 480 examples seen ≈ 1.6 passes over your 300 pairs. For style transfer that's plenty — more epochs on a small dataset starts memorizing individual answers instead of learning the pattern. `adamw_8bit` quantizes optimizer state to fit the T4; `2e-4` is the standard LoRA learning rate (10–100x higher than full fine-tuning, because you're training tiny fresh matrices, not nudging pretrained ones).

**What to verify:** a step-by-step loss table. Loss should start around 2–3 and fall to roughly 1.0–1.5 by step 60. Falling below ~0.5 means it's memorizing — fine to note, but stop there. Expect ~12 minutes on a T4. If loss stays flat, the chat template step went wrong — check Cell 4's output.

## 12. Cell 7 — Talk to Your Fine-Tuned Model

The adapters are now trained, and the same `ask()` function routes through them automatically. Ask the model something that is **not in the training data** — the point of fine-tuning is generalizing the style, not replaying it.

```

tuned_answers = [ask(q) for q in heldout_questions]

for q in [heldout_questions[0], "Why do we split data into train and test set

```

```
s?"]]:
    print("Q:", q)
    print("DESI TUTOR:", ask(q)[:400])
    print("-" * 60)
```

**What to verify:** Hinglish, one desi analogy, "Bottom line:" ending — with **no system prompt**. Put Cell 5's output next to this one. Same weights count, same GPU, same question — completely different model behaviour. That's 60 steps of LoRA.

## 13. Cell 8 — Prove It Beats the Base Model

"It looks better" doesn't survive a code review. This cell makes the claim measurable: for each of the 20 held-out questions, the Groq teacher judges — base answer vs tuned answer — which better matches the DesiTutor spec. Answer order is flipped on every other question so the judge can't develop a position habit (LLM judges measurably favour the first option otherwise).

```
JUDGE_PROMPT = """You are judging which reply better matches the 'DesiTutor' persona:
(1) natural Hinglish, (2) exactly one desi analogy, (3) ends with 'Bottom line:'.
Judge persona adherence only, not factual depth.
Return JSON: {"winner": "A" or "B"}"""

async def judge_one(question: str, base: str, tuned: str, flip: bool) -> bool:
    """Return True if the tuned answer wins. 'flip' swaps A/B order to cancel position bias."""
    a, b = (tuned, base) if flip else (base, tuned)
    resp = await client.chat.completions.create(
        model = "openai/gpt-oss-120b",
        messages = [{"role": "system", "content": JUDGE_PROMPT},
                    {"role": "user", "content": f"Question: {question}\n\nReply A: {a}\n\nReply B: {b}"}],
        response_format = {"type": "json_object"},
        temperature = 0.0,
    )
    winner = json.loads(resp.choices[0].message.content)["winner"]
    return (winner == "A") if flip else (winner == "B")

async def run_eval() -> list[bool]:
    """Judge all held-out questions, 5 at a time."""
    wins = []
    for i in range(0, len(heldout_questions), 5):
        batch = [judge_one(heldout_questions[j], base_answers[j], tuned_answers[j], flip = j % 2 == 1)
                  for j in range(i, min(i + 5, len(heldout_questions)))]
        wins.extend(await asyncio.gather(*batch))
    return wins
```

```
wins = await run_eval()
print(f"Fine-tuned model win rate: {sum(wins)}/{len(wins)} ({100 * sum(wins) / len(wins):.0f}%)")
```

**What to verify:** expect a win rate of 85–100%. Anything at or below 60% means training under-delivered — usually a dataset problem (check Cell 2 samples) or a template problem (check Cell 4). This number is your resume line: *"fine-tuned model beat base model with a 95% judged win rate on a held-out set."*

## 14. Cell 9 — Save the Adapters and Export GGUF

Three save formats, three purposes. The **LoRA adapter** is tiny (~200 MB) and is what you version-control or upload to HuggingFace — anyone can apply it on top of the public base model. The **merged 16-bit** folds adapters into the base weights for serving frameworks like vLLM. The **GGUF** is the quantized single-file format that llama.cpp and Ollama run on ordinary laptops.

```
model.save_pretrained("desitutor_lora")
tokenizer.save_pretrained("desitutor_lora")
print("LoRA adapters saved.")
```

The GGUF export is optional and the slowest step — Kaggle compiles llama.cpp from source the first time, which adds ~10 minutes. `q4_k_m` is the standard 4-bit quantization: the 4B model becomes a ~2.5 GB file.

```
model.save_pretrained_gguf("desitutor_gguf", tokenizer, quantization_method = "q4_k_m")
```

**What to verify:** the Output panel (right side) shows `desitutor_lora/` and a `.gguf` file under `desitutor_gguf/`. Click **Save Version** → **Save & Run All** if you want Kaggle to persist outputs beyond the session, or just download the files directly from the Output panel. Download the GGUF before the session dies — sessions cap at 12 hours and idle sessions get reclaimed.

## 15. Run It Locally with Ollama

Back on your own machine, with [Ollama](#) installed: put the downloaded `.gguf` in a folder and create a file named `Modelfile` (no extension) next to it:

```
FROM ./Qwen3-4B-Instruct-2507.Q4_K_M.gguf

TEMPLATE """{{ if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{ if .Prompt }}<|im_start|>user
{{ .Prompt }}<|im_end|>
{{ end }}<|im_start|>assistant
"""

PARAMETER stop "<|im_end|>"
```

The `TEMPLATE` block is Ollama's version of the same ChatML format from Cell 4 — the file name of your GGUF may differ slightly, so check what actually downloaded. Then:

```
ollama create desitutor -f Modelfile
ollama run desitutor "What is a vector database?"
```

Your own fine-tuned model, running fully offline on your laptop, answering in Hinglish. Dataset → adapters → GGUF → local inference: the complete production fine-tuning pipeline, and the total bill was ₹0.

## 16. Common Errors & Fixes

| Error                                                                                           | Cause                                                                                                              | Fix                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>xFormers can't load C++/CUDA extensions</code> or <code>undefined symbol</code> on import | pip upgraded/downgraded torch after xformers was installed — they must be compiled against each other              | Restart the session (Run → Restart & clear cell outputs) and run Cell 1 exactly as written, first, once. Never <code>pip install</code> anything after it             |
| <code>RuntimeError: No CUDA GPUs are available</code>                                           | Notebook is running on CPU                                                                                         | Settings → Accelerator → <b>GPU T4 ×2</b> , then rerun from Cell 1 (changing accelerator restarts the session)                                                        |
| <code>ConnectionError</code> / <code>MaxRetryError</code> downloading the model                 | Notebook internet is off                                                                                           | Settings → Internet → On (requires phone-verified account)                                                                                                            |
| <code>ModuleNotFoundError: No module named 'groq'</code>                                        | Kaggle's base image doesn't include the Groq SDK                                                                   | The <code>!pip install groq</code> line in Cell 1 covers it — rerun Cell 1 if you skipped or edited it                                                                |
| <code>SecretNotFoundError: GROQ_API_KEY</code>                                                  | Secret not created or not attached                                                                                 | Add-ons → Secrets → add <b>GROQ_API_KEY</b> and <b>tick the attach checkbox</b> for this notebook                                                                     |
| <code>model_decommissioned</code> from Groq                                                     | Using <code>llama-3.3-70b-versatile</code> — shut down 2026-08-16                                                  | Use <code>openai/gpt-oss-120b</code>                                                                                                                                  |
| <code>429 rate_limit_exceeded ... tokens per minute (TPM): Limit 8000</code>                    | Free tier allows 8,000 tokens/min; each 20-pair call uses ~3,300, so more than 2 concurrent calls always overflows | Handled by the retry loop; if it keeps happening, drop concurrency from 2 to 1 in <code>build_dataset</code>                                                          |
| <code>400 json_validate_failed</code> with empty <code>failed_generation</code>                 | Transient Groq JSON-mode failure — the model's output didn't validate                                              | Handled by the retry loop; the repeat call succeeds                                                                                                                   |
| <code>TypeError: SFTTrainer.__init__() got an unexpected keyword argument</code>                | Unpinned <code>trl</code> — the API differs across versions                                                        | Rerun Cell 1; it pins <code>trl==0.22.2</code> , which this code targets                                                                                              |
| <code>CUDA out of memory</code> during training                                                 | Sequences + batch don't fit in 16 GB                                                                               | Set <code>per_device_train_batch_size = 1</code> and <code>gradient_accumulation_steps = 8</code> (same effective batch), or drop <code>max_seq_length</code> to 1024 |

| Error                                         | Cause                                                           | Fix                                                                                                                               |
|-----------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Loss stays flat around its starting value     | Chat template not applied — model is training on malformed text | Check Cell 4's printed sample for `<`                                                                                             |
| Cell 1 "succeeds" but imports fail afterwards | <code>%%capture</code> swallowed a real pip failure             | Rerun Cell 1 without <code>%%capture</code> and read the actual error                                                             |
| GGUF export hangs for many minutes            | Kaggle is compiling llama.cpp from source                       | Normal — first export takes ~10 min. Skip the GGUF cell if you only need the adapters                                             |
| <code>KeyError: 'examples'</code> in Cell 2   | Teacher returned JSON in a different shape                      | Rare with JSON mode; the retry loop re-asks. If persistent, print <code>resp.choices[0].message.content</code> and adjust the key |

## 17. Knowledge Check

1. Why does the training data need zero system prompts, and where does the persona live after training?
2. Freshly attached LoRA adapters don't change the model's outputs at all. Why? (Hint: how is `B` initialized?)
3. Your dataset is 300 examples and you train for 300 steps at effective batch 8. What goes wrong?
4. Why does `train_on_responses_only` improve results on conversation data?
5. You want the model to know your company's leave policy. Fine-tuning or RAG — and why?

(Answers: 1 — in the weights, via the assistant turns it was trained on. 2 — `B` starts at zero so  $\Delta W = B \cdot A = 0$ . 3 — 2400 examples seen = 8 epochs on 300 pairs; it memorizes answers instead of learning the style. 4 — loss is computed only on assistant tokens, so capacity isn't wasted predicting user questions. 5 — RAG; that's knowledge, which changes and needs citations, not behaviour.)

## 18. What You Should Remember

- **Fine-tune for behaviour, RAG for knowledge.** Style, format, tone, persona → weights. Facts → retrieval.
- **The dataset is the product.** 300 clean, on-spec examples beat 10,000 sloppy ones. Spend your effort on the generation prompt, not the trainer knobs.
- **QLoRA = frozen 4-bit base + trainable 16-bit adapters.** That one sentence is why a 4B model trains on a free 16 GB GPU.
- **Distillation is the standard pipeline.** Big free teacher (gpt-oss-120b) writes data; small student (Qwen3-4B) learns it; the student runs on a laptop.
- **Always keep a held-out set and measure.** A judged win rate turns "it feels better" into a number you can put on a resume.
- **Pin your training stack.** torch/xformers/trl version drift is the #1 cause of dead fine-tuning notebooks.

## 19. Key Terms Learned

| Term                   | Meaning                                                                               |
|------------------------|---------------------------------------------------------------------------------------|
| PEFT                   | Parameter-Efficient Fine-Tuning — any method that trains a small subset of parameters |
| LoRA                   | Low-Rank Adaptation — trainable thin matrices $B \cdot A$ added to frozen weights     |
| QLoRA                  | LoRA with the frozen base quantized to 4-bit                                          |
| Rank (r)               | Width of the LoRA matrices — capacity of the adapter                                  |
| SFT                    | Supervised Fine-Tuning — training on prompt → response pairs                          |
| Distillation           | A large teacher model generating training data for a small student model              |
| Chat template          | The exact token markup a model expects conversations in (ChatML for Qwen)             |
| Response-only training | Masking loss on user turns so only assistant tokens are learned                       |
| Held-out set           | Examples excluded from training, used to measure generalization                       |
| GGUF                   | Single-file quantized model format for llama.cpp / Ollama                             |
| Adapter merging        | Folding LoRA weights into the base model for standalone deployment                    |

## 20. What to Build Next

1. **Your own domain.** Swap the 16 topics and the persona spec — a legal-notice explainer, a GST-filing assistant, your startup's support tone. The pipeline doesn't change; only Cell 2's prompt does.
2. **Scale the dataset and measure the curve.** Rerun with 100 vs 300 vs 1000 examples and plot win rate against dataset size — you'll see with your own eyes where data stops paying.
3. **Publish the adapter.** Push `desitutor_lora` to HuggingFace Hub ( `model.push_to_hub` + a free HF token) so anyone can load your fine-tune on top of the public base model — a public artifact recruiters can actually click.